A New Approach to Large Party

Beaver-Style MPC

with Small Computational Overhead



Aayush Jain

Carnegie Mellon University



Huijia Lin

University of Washington
Google



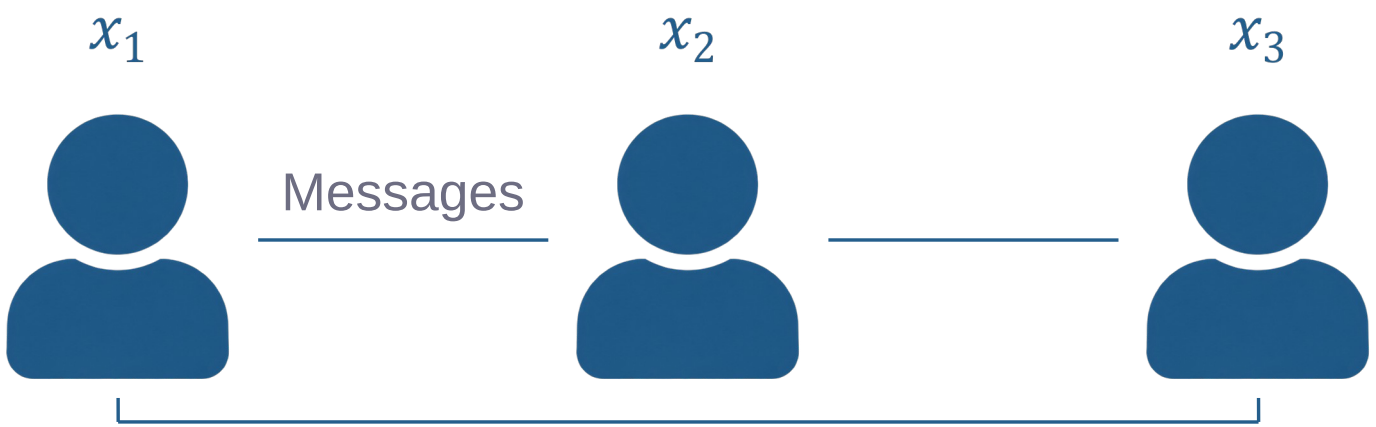
Nuozhou Sun

Carnegie Mellon University

TCC

Multi-party computation

Joint computation on private inputs



Common output

$$y = f(x_1, x_2, x_3)$$

Inputs remain private beyond what the output reveals.

Communication is often the main cost.

Computation can dominate on fast networks
or become restrictive on processors with limited capacity.

Approaches to multi-party computation

N parties jointly compute while keeping their inputs private.

GMW	Evaluate multiplication gates using pairwise oblivious transfers.
Beaver-style MPC	Generate triples using OT, PCG/PCF, or HE. Use them in a cheap online phase.
HSS, FHE, iO	Local evaluation on shares, encrypted computation, or obfuscation-based protocols.

Computation overhead of existing approaches

Semi-honest MPC: amortized cost per party and Boolean gate

Approach	Computation
GMW / triples via OT	$O(N)$
Programmable PCG/PCF triples	$O(N)$
HE-generated triples	$\mathrm{polylog}(\lambda + N)$
HSS-based MPC*	$\mathrm{poly}(\lambda)$
FHE-based MPC*	$\mathrm{poly}(\lambda)$
iO-based MPC	$\mathrm{poly}(\lambda)$

Beaver online phase: $O(1)$, given multiplication triples

*Special cases can be cheaper. N: parties. λ: security parameter.

Outline of our scheme

Noisy Beaver triples

Cheap local generation

+

Noise-tolerant online phase

Correct errors and protect privacy

MPC with small computational overhead

Our objective and result

Small computation overhead, with low communication

Semi-honest MPC with up to $N - 1$ corruptions

Computation per party / gate

$$O(k^2 c(\lambda))$$

Communication per party / gate

$$O(c(\lambda))$$

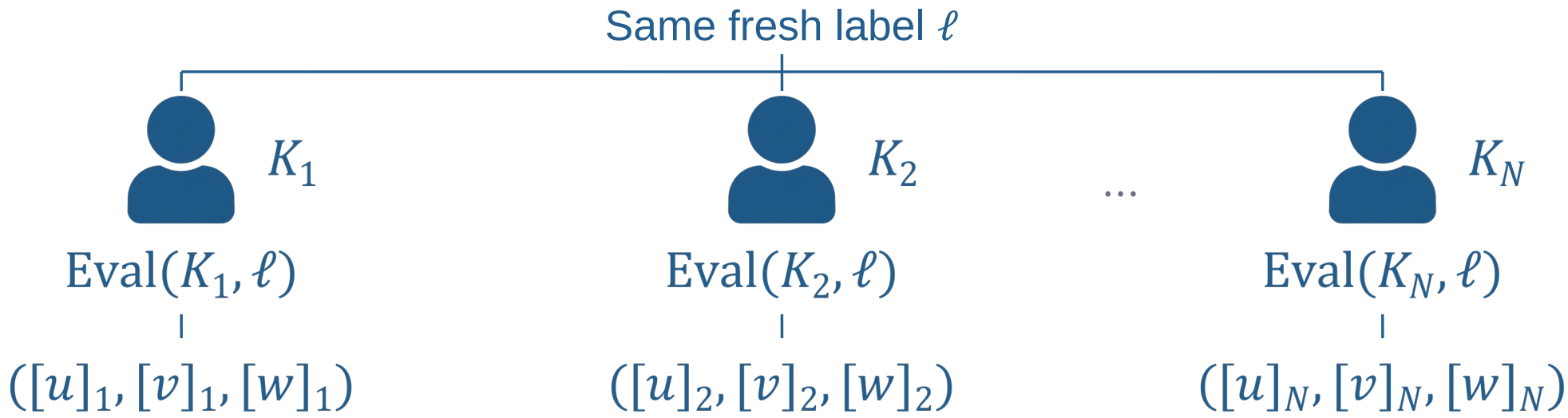
$c(\lambda)$ grows arbitrarily slowly. No dependence on N.

Low-noise k-Sparse LPN, random oracle model, reusable setup

Arbitrarily slow k gives arbitrarily small super-constant overhead.

Noisy pseudorandom correlation functions

Reusable keys. Local evaluation without interaction.



Almost correct $\Pr[w \neq uv] \leq \delta$

Pseudorandom Like random Beaver shares, plus a rare noise vector.

Standard LPN and Sparse LPN

Noisy linear equations hide a fixed secret s.

$$(a, \langle a, s \rangle + e) \approx_c (a, r)$$

	Standard LPN	**k-Sparse LPN**
Public vector a	Uniform in $F_2{}^n$	Exactly k nonzero entries
Work for $\langle a,s \rangle$	$O(n)$	$O(k)$

e is Bernoulli noise. r is a fresh uniform bit.

We assume hardness in the low-noise sparse regime.

A noisy quadratic correlation

Choose secret vectors s, t. Let $Q = s \otimes t$.

$$u = \langle a, s \rangle + e_1$$

$$v = \langle a, t \rangle + e_2$$

$$w = \langle a \otimes a, Q \rangle + e_3 = \langle a, s \rangle \langle a, t \rangle + e_3$$

Security under low-noise Sparse LPN

$$(a, u, v, w) \approx_c (a, r_1 + e_1, r_2 + e_2, r_1 r_2 + e_3)$$

Fresh uniform r_1, r_2 and the chosen correlated noise.

Security from Sparse LPN

Fix t $\quad X = \langle a, s \rangle, \qquad b = \langle a, t \rangle \in \{0, 1\}$

$$(u, v, w) = (X + e_1,\ b + e_2,\ bX + e_3)$$

For intuition: independent $e_1, e_3 \sim \mathrm{Ber}(\delta)$.

$$b = 0 : (u, w) = (X + e_1, e_3)$$

$$b = 1 : (u, w) = (X + e_1, X + e_3)$$

Sparse LPN at noise rate δ^2 reduces to distinguishing these views.

Local evaluation for N parties

Party i holds additive shares $[s]_i$, $[t]_i$, $[Q]_i$.

A random oracle gives the common k-sparse vector a.

$$u_i = \langle a, [s]_i \rangle + e_{1,i}$$

$$v_i = \langle a, [t]_i \rangle + e_{2,i}$$

$$w_i = \langle a \otimes a, [Q]_i \rangle + e_{3,i}$$

$a \otimes a$ has k^2 nonzero entries: $O(k^2)$ work per party.

$$e_1 = \sum_{i=1}^{N} e_{1,i}$$

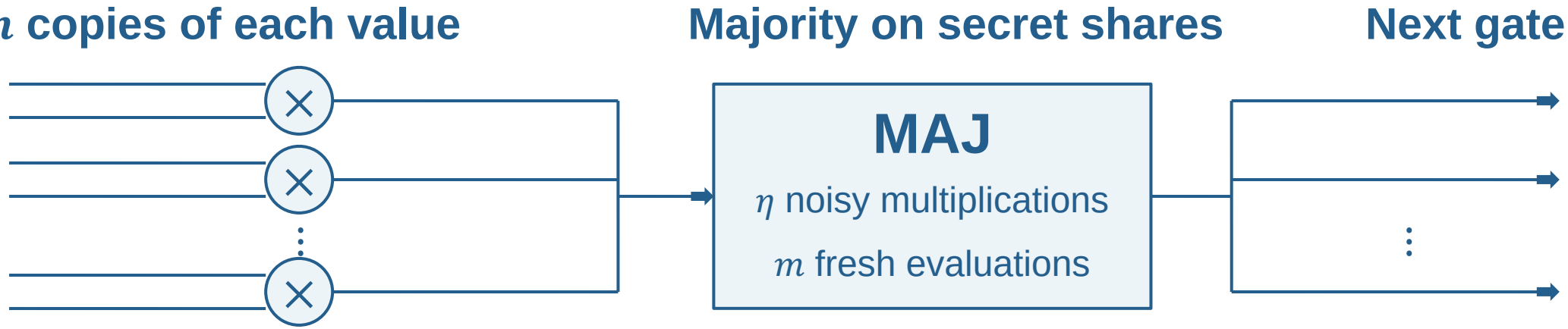
Each $e_{1,i}$ is Bernoulli with a lower probability.
Local rate $O(\delta/N)$ keeps global noise small.

Sums are over F_2. Each party's three noise coordinates are correlated.

Noisy triples to MPC

Remaining task: evaluate any circuit using noisy triples.

Compile the circuit to tolerate noisy gates.



Repeat after every gate.

Extra majority layers protect the final opening.

The paper also gives a more efficient conversion.

Any superconstant overhead

The number of copies can grow arbitrarily slowly.

$$m(\lambda) \in \omega(1)$$

Negligible error under inverse-polynomial gate noise.

For every $h(\lambda) \in \omega(1)$,

$$\mathrm{Comp},\ \mathrm{Comm} = O(h(\lambda))$$

Per party, per gate, after reusable setup.
Assuming low-noise Sparse LPN at the chosen sparsity.